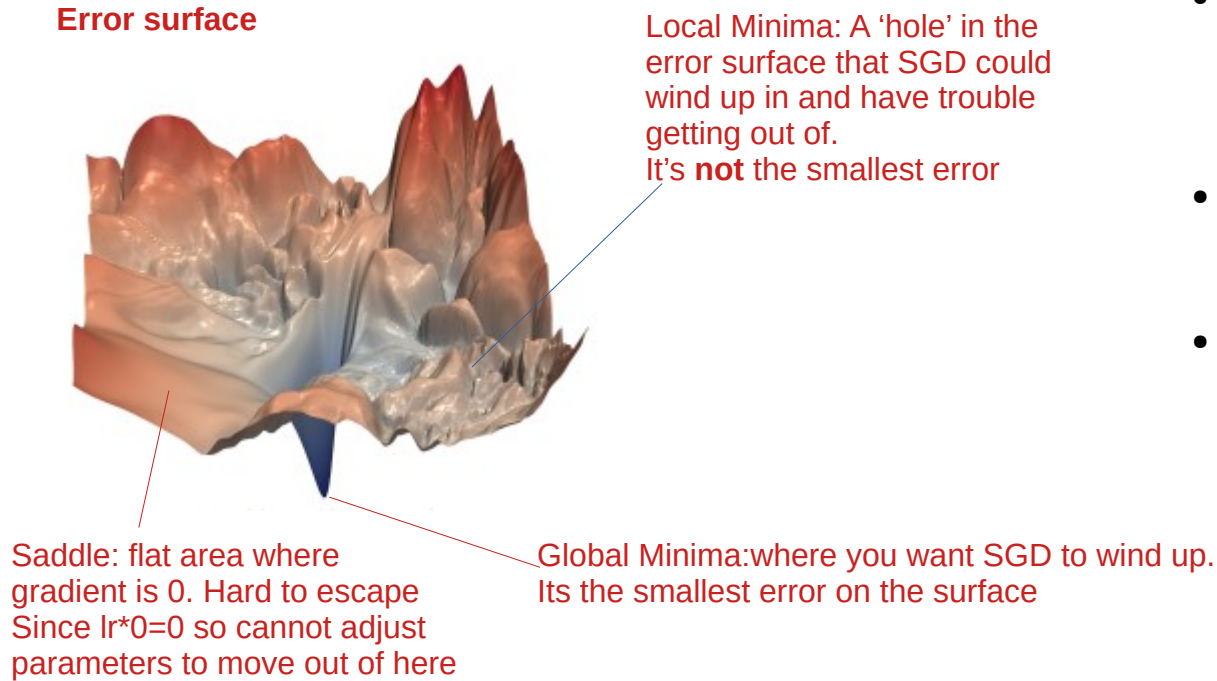


Optimizers- Gradient Descent (GD) verses Adam

Outline

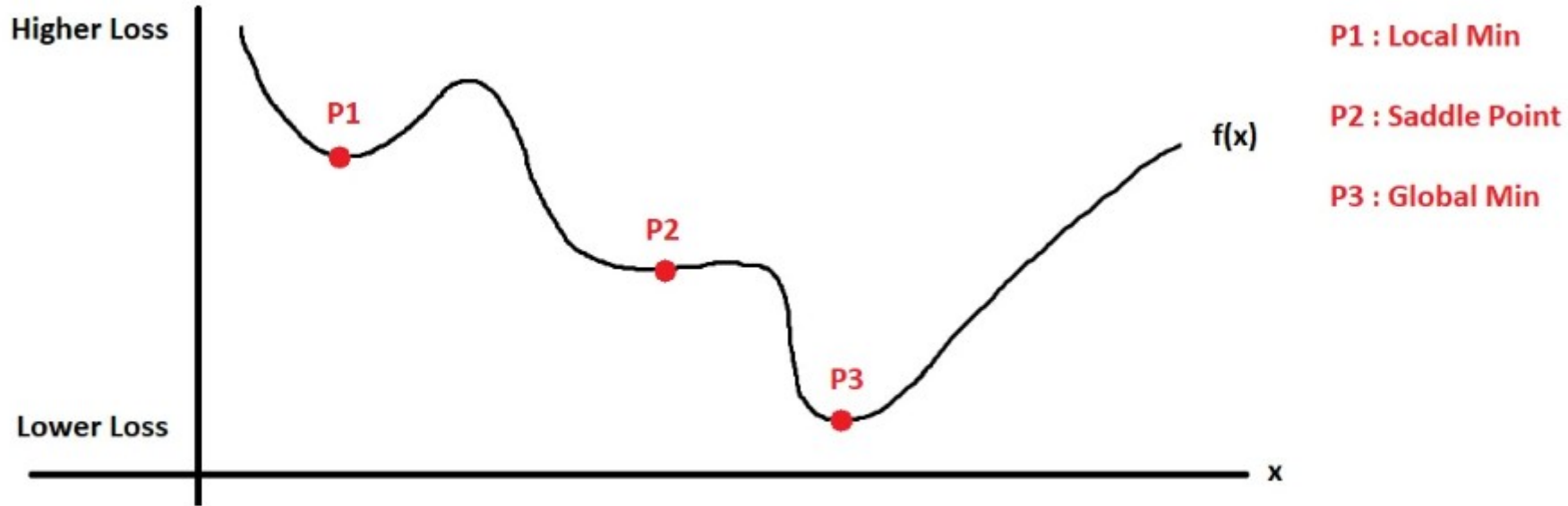
- Error surface
 - Local Minima, saddle, global minima
- Optimizers
 - Gradient Descent
 - Adam
- Tradeoffs
- Further Reading

Error Surface



- Projected error surface of a Resnet 54 CNN for 1 batch of data
- Millions of parameters projected into 3d space
- Want to get the error to the global minima, but might get stuck in a saddle or local minima with GD

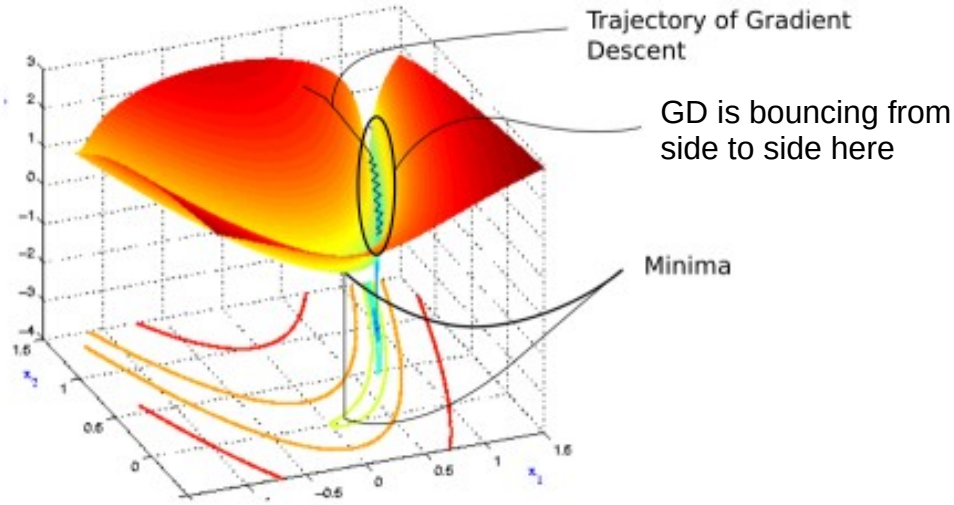
Error Surface



Optimizer: Gradient Descent

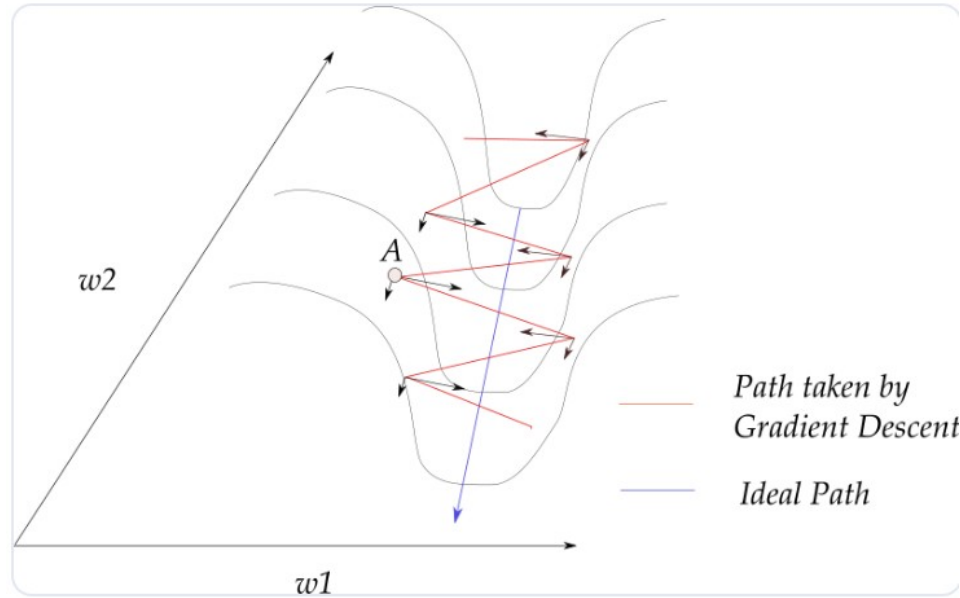
- GD analogy:
 - Analogy: A person taking equal-sized steps straight downhill
 - Updates the model params to reduce error
 - You get there but might get stuck in local minima or a saddle
 - Mini batch GD helps
 - every batch has different error surface
 - Even if stuck for this batch (no param update) will probably not be stuck for the next batch (param update)
 - See **Gradient Descent**

Optimizer: Gradient Descent



GD in a narrow 'canyon' will overshoot when params are updated and will 'bounce' from side to side

Optimizer: Gradient Descent with Momentum



- Movements decomposed into 2 directions
- If we add these vectors, the zig zag ones cancel out
- Left with vectors pointing in correct direction

Optimizer: Adam-momentum

- Momentum using an exponential decay
- a fraction of previous updates added to current update
- Increases update in direction of descent
- Helps to smooth noisy gradients and get thru small dips and saddles
- **Analogy: Pushing the person walking down the hill in the right direction**

Optimizer: Adam-adaptive step size

- If high gradient variance (big curvature in error surface) then shorten step size
- If not, lengthen step size
- Effect: Take smaller steps in steep/unstable directions and bigger steps in gentle/flat directions, reducing zig-zag in ravines

Optimizer: Adam- Why its popular

- Stability – out of the box params work well
- Faster early progress: Momentum + adaptivity helps escape plateaus/saddles quickly.

Adam: Tradeoffs

- Generalization: On some vision tasks, SGD + momentum + a good LR schedule can match or even beat Adam's final test accuracy.
- Not a silver bullet: Truly dead ReLUs, terrible data scaling, or wild learning rates will still cause trouble.

Other Optimizers

- AdamW- has weight decay, better regularization
- RMSProp – Adam derives partly from this
- Adagrad – not covered here
- Adadelta – “

Optimizers where in Pytorch

```
1 lr=1e-2
2 criterion = nn.MSELoss() #dont need to create your own loss function, pytorch has many built in
3 optimizer = torch.optim.SGD(model.parameters(), lr=lr) #tracks the parameters of model and updates them
4 # optimizer = torch.optim.Adam(model.parameters(), lr=1e-3) #faster optimizer
5
```

Declare an optimizer to use

In the training loop

```
# 1) Reset gradients from the previous step
optimizer.zero_grad(set_to_none=True)
# set_to_none=True can be a tiny memory/perf win vs. zeroing to 0.0
```

Zero the gradient for all model parameters

```
# 2) Forward pass: compute predictions and loss
preds = model(xb)
loss = criterion(preds, yb) # e.g., MSE loss
```

Calculate the loss

```
# 3) Backward pass: compute gradients w.r.t. parameters
loss.backward()
```

Backpropagate to get the gradients

```
# 4) Update parameters using the optimizer, this will
# subtract a small bit (lr*grad) of the gradient from each parameter
optimizer.step()
```

Adjust the model parameters ($p - lr * grad$)

Further Reading

For further detail see;

Equations behind GD, Momentum and Adam

Optimizer Visualizers